

Name:Atharv revat shukla

PRN:25070122053

Division:A2

Subject: Programming with Java

ASSIGNMENT No :3

TITLE : develop a Java application demonstrating method overloading and the use of static variables, methods in class design.

Problem Statement

Develop a Java application demonstrating method overloading and the use of static variables, methods in class design.

Learning Objectives

To implement method overloading and use static variables and methods in Java programs.

Theory

Method overloading allows multiple methods to have the same name with different parameter lists,

enabling compile-time polymorphism. The static keyword is used for variables and methods that

belong to the class instead of individual objects. Static members share common data among all

objects and reduce memory usage. These concepts improve code flexibility, readability, and efficient resource management.

Outcome

Conclusion

Exercise

1. Develop a Calculator program using overloaded methods for addition of integers and decimals. Use a static variable to count calculations.

```
public class calculator {  
    private static int calculation_count = 0;  
    public int add(int a, int b) {  
        calculation_count++;  
        return a + b;  
    }  
    public int add(int a, int b, int c) {  
        calculation_count++;  
        return a + b + c;  
    }  
    public double add(double a, double b) {  
        calculation_count++;  
        return a + b;  
    }  
    public double add(double a, double b, double c) {  
        calculation_count++;  
        return a + b + c;  
    }  
    public static int get_calculation_count() {  
        return calculation_count;  
    }  
    public static void main(String[] args) {  
        calculator calc = new calculator();  
        int sum_int_2 = calc.add(5, 10);  
        System.out.println(sum_int_2);  
        int sum_int_3 = calc.add(4, 8, 12);
```

```

        System.out.println(sum_int_3);
        double sum_double_2 = calc.add(5.5, 10.25);
        System.out.println(sum_double_2);
        double sum_double_3 = calc.add(1.1, 2.2, 3.3);
        System.out.println(sum_double_3);
        System.out.println(calculator.get_calculation_count());
    }
}

```

2. Develop a Restaurant Billing Application where overloaded methods calculate bills for dine-in, takeaway, and delivery orders, while static variables track total orders.

```

public class bill_system {
    static int total_orders = 0;
    static double total_money = 0;
    public double make_bill(double food_price) {
        total_orders++;
        double tax = food_price * 0.05;
        double final_bill = food_price + tax;
        total_money += final_bill;
        return final_bill;
    }
    public double make_bill(double food_price, double packing_fee) {
        total_orders++;
        double tax = food_price * 0.05;
        double final_bill = food_price + packing_fee + tax;
        total_money += final_bill;
        return final_bill;
    }
}

```

```
}

public double make_bill(double food_price, double distance, double tip) {

    total_orders++;

    double tax = food_price * 0.05;

    double delivery_fee = distance * 1.5;

    double final_bill = food_price + delivery_fee + tip + tax;

    total_money += final_bill;

    return final_bill;

}

public static void main(String[] args) {

    bill_system shop = new bill_system();

    double order_1 = shop.make_bill(25.0);

    System.out.println("dine in bill: " + order_1);

    double order_2 = shop.make_bill(18.5, 2.0);

    System.out.println("takeaway bill: " + order_2);

    double order_3 = shop.make_bill(30.0, 4.5, 5.0);

    System.out.println("delivery bill: " + order_3);

    System.out.println("total orders today: " + total_orders);

    System.out.println("total cash made: " + total_money);

}

}
```